

feature

Base class for all Plan Features.

LOGGER module-attribute

```
LOGGER = getLogger(__name__)
```

BaseFeature

```
BaseFeature(name: str, description: Optional[str] = None, operator: Optional[BaseFeaturesOperator] = None)
```

Bases: `ABC`

Abstract base class for all Features in a Plan.

Parameters:

Name	Type	Description	Default
<code>name</code> 	<code>str</code>	Name of the feature	<i>required</i>
<code>description</code> 	<code>str</code>	description of the feature.	<code>None</code>
<code>operator</code> 	<code>BaseFeaturesOperator</code>	Operator where the feature is in.	<code>None</code>

`description` property writable

```
description
```

`name` property writable

```
name
```

`operator` property

```
operator
```

`from_dict` abstractmethod classmethod

```
from_dict(feat_dict: Dict[str, str]) -> BaseFeature
```

Creates a Feature from a dict type to facilitate importing features from a tabular format.

Parameters:

Name	Type	Description	Default
<code>feat_dict</code> 	<code>dict</code>	Dictionary containing the feature in a tabular format.	<i>required</i>

Returns:

Type	Description
<code>BaseFeature</code>	Feature initiated with the attributes from the dict.

from_java_feature `abstractmethod` `classmethod` [¶](#)

```
from_java_feature(feature: JavaClass, operator: BaseFeaturesOperator) -> BaseFeature
```

Converts a Java DS-API Feature to a Python feature class.

get_dependencies `abstractmethod` [¶](#)

```
get_dependencies() -> Set[str]
```

Gets the list of other features names that are used to create the feature.

Dependencies are not recursive.

Returns:

Type	Description
<code>set</code>	Set of feature names.

get_dependencies_by_type `abstractmethod` [¶](#)

```
get_dependencies_by_type() -> Dict[str, List[str]]
```

Gets the lists (by type of dependency) of other features names that are used to create the feature.

Dependencies are not recursive.

Returns:

Type	Description
<code>Dict[str, List[str]]</code>	Dictionary of dependencies, the keys are the dependency types and the values are the list of feature names.

rename_dependencies `abstractmethod` [¶](#)

```
rename_dependencies(old_name: str, new_name: str)
```

Renames the feature dependencies matching the old name.

Parameters:

Name	Type	Description	Default
<code>old_name</code> ¶	<code>str</code>	Old name to change from.	<i>required</i>
<code>new_name</code> ¶	<code>str</code>	New name to change to.	<i>required</i>

to_dict `abstractmethod` [¶](#)

```
to_dict(with_operator: bool = True) -> dict[str, str | float | int | bool | None]
```

Converts the feature to a dict type to facilitate exporting features into a tabular format.

Parameters:

Name	Type	Description	Default
<code>with_operator</code> ¶	<code>bool</code>	When 'false', returns the object without its operator attributes.	<code>True</code>

Returns:

Type	Description
<code>dict</code>	Dictionary containing all the editable fields from the plan to create this feature.

Raises:

Type	Description
<code>AttributeError</code>	When <code>with_operator</code> is true and the feature has not been added to an operator.

`to_java_feature_builder` [abstractmethod](#) [¶](#)

```
to_java_feature_builder() -> JavaClass
```

Converts a Python feature class to a Java DS-API Feature Builder.